Haystack Dependency Injection Documentation

Introduction

The Haystack Dependency Injection package simplifies managing dependencies in Unity projects. This guide provides an overview of setup, key attributes, and example implementations.

Key Attributes

- **`[Haystack.Inject]`**: Marks fields to be injected with dependencies.
- **`[Haystack.Install]`**: Marks fields to be installed with dependencies.
- **`[Haystack.DirectHayClass]`**: Indicates that the class uses direct injection with tags or keys.
- **`[Haystack.PhysicsClass]`**: Specifies that the class handles physics-related injections.

Direct Injection Overview

Direct Injection in Haystack allows you to specify dependencies directly in your Unity scripts using tags or keys. This method is straightforward and useful for cases where you need to explicitly define which dependencies should be injected into your components.

Installing Dependencies

Create a script that installs dependencies using a tag or key.

```
[Haystack.DirectHayClass]
public class SampleInstaller : MonoBehaviour
{
    [SerializeField][Haystack.Install("p1")] SampleDependency _sample;

    [HayAwaker]
    protected void Awaker()
    {
        Debug.Log("Dependencies Installed.");
    }
}
```

In this example:

- `SampleInstaller` installs `SampleDependency` using the tag `"p1"`.

3. Injecting Dependencies

Create a script that uses direct injection to access the installed dependencies.

```
[Haystack.DirectHayClass]
public class SampleInjector : MonoBehaviour
{
    [SerializeField][Haystack.Inject("p1"] SampleDependency _sample;

    [HayStarter]
    protected void Starter()
    {
        Debug.Log("Dependencies Injected.");
    }
}
```

In this example:

- `SampleInjector` uses `SampleDependency` injected with the tag `"p1"`.
- The `Starter` method updates UI elements with the injected dependency data.

IndirectHayBuilder Overview

`IndirectHayBuilder` is a utility class in the Haystack Core Dependency Injection framework that facilitates the installation and injection of dependencies using indirect references, such as custom keys. It enables flexible dependency management by allowing you to dynamically specify which dependencies to inject or install based on keys or tags.

Key Functions

1. **`InjectAll`**:
    - **Purpose**: Injects all dependencies into a target object based on a specified key.
    - **Usage**: Useful when you need to dynamically select and inject a set of dependencies into a component at runtime.
    - **Example**:
      ```
      public void SetToPlayer1()
      {
          _indirectHayBuilder.InjectAll(this, _player1DepKey);
      }
      ```
      In this example, `InjectAll` is used to inject dependencies into the current object (`this`) using the key `_player1DepKey`.
2. **`InstallAll`**:
    - **Purpose**: Installs all dependencies into a target object based on a specified key.
    - **Usage**: Used to initialize and set up dependencies for a component using a key or tag, typically during the object's setup phase.

- **Example**:

```
void Awake()
{
    _indirectHayBuilder.InstallAll(this, _customKey);
}
```

Here, `InstallAll` is used to install dependencies into the current object (`this`) based on the `_customKey`.

How to Use `IndirectHayBuilder`

1. **Define Dependencies**:
   - Create classes or structs to define the dependencies you need. Ensure they are marked with appropriate attributes like `[Haystack.Install]` for installation.

```
[Serializable]
public struct CustomKDependency
{
    public string Name;
    public string Description;
}
```

2. **Set Up Dependency Installation**:
   - Use `IndirectHayBuilder` to install dependencies into your components. This involves specifying a custom key that maps to the dependencies you want to inject.

```
public class CustomPkVarient1 : MonoBehaviour
{
    [SerializeField] string _customKey;
    [Haystack.Install] [SerializeField] protected CustomKDependency
_customKDependency;
    readonly IndirectHayBuilder _indirectHayBuilder = new();

    void Awake()
    {
        _indirectHayBuilder.InstallAll(this, _customKey);
    }
}
```

In this example, `InstallAll` is used to install dependencies into `CustomPkVarient1` using `_customKey`.

3. **Inject Dependencies Dynamically**:
   - Use `IndirectHayBuilder` to inject dependencies into components based on keys or tags, often in response to runtime conditions.

```
public class CustomKManager : MonoBehaviour
{
    [SerializeField] string _player1DepKey;
    [SerializeField] string _player2DepKey;
    [Haystack.Inject][SerializeField] protected CustomKDependency
_customKDependency;
    readonly IndirectHayBuilder _indirectHayBuilder = new();

    public void SetToPlayer1()
    {
        _indirectHayBuilder.InjectAll(this, ()=>
        {
            //Callback after injected all
        }, _player1DepKey);
    }

    public void SetToPlayer2()
    {
        _indirectHayBuilder.InjectAll(this, ()=>
        {
            //Callback after injected all
        }, _player2DepKey);
    }
}
```

In this example, `InjectAll` dynamically injects dependencies into `CustomKManager` based on `_player1DepKey` or `_player2DepKey`.

Getting Dependencies Through Colliders

Collider Attributes

- **`[Haystack.Install(Haystack.ColliderType.Collider3D)]`**: Marks fields to be installed with dependencies.
- **`[Haystack.Install(Haystack.ColliderType.Collider2D)]`**: Marks fields to be installed with dependencies.

Using Collider Overlap

To retrieve dependencies through colliders using an overlap sphere, you can use the following code in your `Update` method:

```
private void Update()
{
    var near = Physics.OverlapSphere(transform.position, _range, _layerMask);

    foreach (var collider in near)
    {
        collider.InjectDependency<PlayerDependencies>(this, dep =>
        {
            // Other code here…
        });
    }
}
```

Using Trigger Enter

To retrieve dependencies when an object enters a trigger collider, use the following code in your `OnTriggerEnter` method:

```
private void OnTriggerEnter(Collider other)
{
    other.InjectDependency<PlayerDependencies>(this, dep =>
    {
        // Other code here…
    });
}
```

Direct Installation

You can directly install dependencies using the `Haystack.DirectHayClass` attribute:

```
[Haystack.DirectHayClass]
public class SamplePlayer : MonoBehaviour
{
    [Haystack.Install(Haystack.Component.Collider)] PlayerDependencies
_playerDependencies;
}
```

Indirect Installation

Alternatively, use the `IndirectHayBuilder` for installation:

```
public class SamplePlayer : MonoBehaviour
{
    [Haystack.Install(Haystack.Component.Collider)]PlayerDependencies
_playerDependencies;
    protected IndirectHayBuilder _builder = new();

    private void Awake() => _builder.InstallAll(this);
}
```

Summary of Direct and Indirect Injection

Direct Injection

Definition: Direct injection involves specifying dependencies using explicit tags or keys in Unity scripts. This method is straightforward and uses attributes to mark fields for injection and installation.

Use Cases:

- **Fixed Dependencies**: Ideal for scenarios where dependencies are known ahead of time and can be directly mapped to specific tags or keys.
- **Simpler Setup**: Useful for simpler projects or components where dependencies are straightforward and do not change dynamically.
- **Example**: Assigning UI elements or static game settings that are always required and do not need to be swapped out or configured dynamically.

Benefits:

- **Clarity**: Directly specifies which dependencies are needed, making the setup clear and easy to understand.
- **Ease of Use**: Simple to implement and manage when dependencies are stable and fixed.

Indirect Injection

Definition: Indirect injection uses dynamic references, such as custom keys, to manage dependencies. It allows more flexibility by enabling dependencies to be installed or injected based on runtime conditions or configuration.

Use Cases:

- **Dynamic Dependencies**: Ideal for cases where dependencies need to be selected or changed at runtime based on game states or user input.
- **Complex Configurations**: Useful for more complex setups where dependencies might vary between different scenarios or levels.
- **Example**: Switching configurations or assets depending on the game level or player choice, such as loading different sets of game data or configurations dynamically.

Benefits:

- **Flexibility**: Allows for dynamic configuration and runtime adjustments, making it suitable for more complex game systems.
- **Scalability**: Facilitates managing multiple dependency sets or configurations, adapting to varying requirements as the game progresses.

Application in Game Concepts

- **Direct Injection**: Best for static or consistent components, such as core game settings, UI elements that don't change, or default configurations. It simplifies the setup for components that always require the same dependencies.
- **Indirect Injection**: Ideal for scenarios with dynamic or interchangeable components, such as loading different player profiles, adjusting game difficulty settings, or switching between different asset sets. It provides the flexibility needed for games with variable content or changing conditions.

By using these injection methods effectively, developers can streamline dependency management in their projects, improving maintainability and adaptability.

Unbinding Dependencies

Key Features

1. **Unbinding After Injection**: Dependencies can be unbound immediately after they are injected, ensuring that they are not reused.
2. **Collider-Based Unbinding**: Dependencies can be unbound when triggered by collider events, allowing dynamic control based on gameplay interactions.
3. **Direct Injection Unbinding**: You can unbind dependencies at any time after they have been directly injected.

Sample Code Overview

Sample Unbind Manager (SUManager)

This script demonstrates how to unbind dependencies after injection using the `IndirectHayBuilder` for injecting and managing dependencies.

```
public class SUManager : MonoBehaviour
{
    [Haystack.Inject(Haystack.Bind.UnbindAfterInjection)] protected CustomKDependency
_customKDependency;
}
```

Key Points:

- **Unbinding After Injection**: The `CustomKDependency` is injected with the attribute `[Haystack.Inject(Haystack.Bind.UnbindAfterInjection)]`, ensuring it is unbound immediately after injection.
- **Dynamic Dependency Management**: The methods `SetToPlayer1` and `SetToPlayer2` demonstrate how dependencies can be injected and unbound dynamically during runtime.

Collider-Based Unbinding (SampleCollidersUnbinding)

This script demonstrates how to unbind dependencies using collider events.

```
[Haystack.PhysicsClass]
class SampleCollidersUnbinding : MonoBehaviour
{
    private void OnTriggerEnter(Collider other)
    {
        other.InjectDependency<object>(this, result => { });
        other.Unbind();
    }
}
```

Key Points:

- **Collider-Based Injection and Unbinding**: Dependencies are injected into colliding objects, and immediately unbound using the `Unbind()` method. This is useful for temporary interactions.

Important Notes

- **Global Unbinding Effect**: When a dependency is unbound, all injections of the same dependency type will be affected. Unbound dependencies cannot be reinjected unless they are re-registered.

Conclusion

This package provides a powerful toolset for managing dependencies within Unity projects. The unbinding feature offers flexibility in how and when dependencies are used, making it ideal for scenarios where temporary or dynamic dependency management is needed.